

APIs REST

Juegos en Red - Grado en Diseño y Desarrollo de Videojuegos

Ruben Rodríguez

ruben.rodriguez@urjc.es

URJC

Natalia Madrueño

natalia.madrueño@urjc.es

URJC

Mario Manzano

mario.manzano@urjc.es

URJC

2026-09-09



Tabla de contenidos

- [Introducción a APIs REST](#)
- [Niveles de Madurez REST \(Modelo de Richardson\)](#)
- [Formato JSON](#)
- [Principios de Diseño REST](#)
- [Métodos HTTP](#)
- [Anatomía de Peticiones y Respuestas](#)
- [Códigos de Estado HTTP](#)
- [Servidor con Node.js y Express](#)
- [Cliente JavaScript con fetch\(\)](#)
- [Ventajas de la Arquitectura REST](#)
- [Resumen](#)



Introducción a APIs REST

Contexto

Después de implementar el juego con Phaser 3

- Cliente y servidor necesitan comunicarse
- Gestionar registro y autenticación de jugadores
- Almacenar puntuaciones y estadísticas
- Gestionar partidas y salas de juego
- Consultar rankings y perfiles

Nota

REST para operaciones que no requieren actualización instantánea

Comunicación HTTP Tradicional vs API REST

Aplicaciones web tradicionales:

- Peticiones HTTP devuelven documentos HTML
- El navegador renderiza la página completa

Aplicaciones con AJAX y SPA:

- Peticiones HTTP intercambian información estructurada
- No devuelven HTML, sino datos (típicamente JSON)
- El cliente procesa y actualiza la interfaz

Petición:
GET <http://www.mygame.com/players/alice>

Respuesta:
{
 "id": "alice",
 "name": "Alice Smith",
 "level": 15,
 "score": 8500
}



URI y URL

URI (Uniform Resource Identifier)

- Identificador único de un recurso
- Puede ser un nombre, una localización, o ambos
- Ejemplo: `/players/alice`
- Toda URL es una URI, pero no toda URI es una URL

URL (Uniform Resource Locator)

- Tipo específico de URI
- Indica **cómo acceder** al recurso
- Incluye protocolo y localización
- Ejemplo:
`https://api.game.com/players/alice`

Estructura de una URL:

`https://api.game.com:443/players/alice?sort=score#stats`

Protocolo Dominio Puerto Ruta Query Fragment

- **Protocolo:** `http://` o `https://` (comunicación segura)
- **Dominio:** Nombre del servidor (`api.game.com`)
- **Puerto:** Opcional, por defecto 80 (HTTP) o 443 (HTTPS)
- **Ruta:** Identifica el recurso específico (`/players/alice`)



¿Qué es una API REST?

REST (Representational State Transfer)

- Estilo de arquitectura para sistemas distribuidos
- Los recursos del servidor se representan mediante URLs
- El cliente transfiere representaciones de esos recursos (JSON, XML, etc.)
- El “estado” reside en esas representaciones que se transfieren

Principios REST (para ser RESTful):

1. **Cliente-Servidor:** Separación de responsabilidades
2. **Sin estado (Stateless):** Cada petición contiene toda la información necesaria
3. **Cacheable:** Las respuestas indican si pueden ser cacheadas
4. **Interfaz uniforme:** URLs, métodos HTTP y representaciones consistentes
5. **Sistema en capas:** Arquitectura puede tener intermediarios (proxies, load balancers)

Niveles de Madurez REST (Modelo de Richardson)

Nivel 0 - The Swamp of POX

Plain Old XML

- HTTP solo como sistema de transporte
- Todas las peticiones POST a una única URL
- El cuerpo indica qué operación realizar
- No aprovecha características de HTTP

```
POST /api HTTP/1.1  
Body: <operation>getPlayer</operation><id>alice</id>
```

Advertencia

Similar a usar HTTP como túnel para otros protocolos

Nivel 1 - Resources

Introducción de recursos individuales

- Cada recurso tiene su propia URI específica
- `/players/alice` y `/matches/123` son URIs distintas
- Todavía no usa correctamente los métodos HTTP
- Típicamente solo POST para todas las operaciones

```
POST /players/alice HTTP/1.1  
POST /matches/123 HTTP/1.1
```



Nivel 2 - HTTP Verbs

Uso correcto de métodos y códigos HTTP

- GET para obtener, POST para crear
- PUT para actualizar, DELETE para eliminar
- Códigos de estado apropiados (200, 404, 500, etc.)
- Aprovecha la semántica del protocolo HTTP

! Importante

Este es el nivel que usaremos en este curso

GET /players/alice HTTP/1.1	→ 200 OK
POST /matches HTTP/1.1	→ 201 Created
DELETE /players/bob HTTP/1.1	→ 204 No Content



Nivel 3 - HATEOAS

Hypermedia as the Engine of Application State

- Las respuestas incluyen enlaces hipermedia
- Guían al cliente sobre acciones disponibles
- Los clientes descubren dinámicamente la API

```
1 {  
2   "id": "alice",  
3   "name": "Alice Smith",  
4   "links": {  
5     "matches": "/players/alice/matches",  
6     "friends": "/players/alice/friends"  
7   }  
8 }
```

Nota

Pocas APIs implementan HATEOAS en la práctica debido a su complejidad

Formato JSON

JavaScript Object Notation

Formato estándar para intercambio de datos

- Ligero y fácil de leer para humanos
- Fácil de parsear para máquinas
- Basado en JavaScript pero independiente del lenguaje

Estructuras principales:

Objetos:

- Colección de pares clave-valor
- Encerrados en `{ }`

```
1 {  
2   "nombre": "Alice",  
3   "nivel": 15  
4 }
```

Arrays:

- Lista ordenada de valores
- Encerrados en `[ ]`

```
1 [  
2   "item1",  
3   "item2",  
4   "item3"  
5 ]
```



Tipos de Valores JSON

Los valores pueden ser:

- **Cadenas de texto:** "Hola mundo"
- **Números:** 42 , 3.14 , -10
- **Booleanos:** true , false
- **Null:** null
- **Objetos:** { "key": "value" }
- **Arrays:** [1, 2, 3]



Ejemplo Completo: Estado de Partida

```
1 {  
2   "game": {  
3     "id": "match_12345",  
4     "type": "pong",  
5     "status": "active",  
6     "players": [  
7       {  
8         "id": "alice",  
9         "name": "Alice Smith",  
10        "score": 5,  
11        "ready": true  
12      },  
13      {  
14        "id": "bob",  
15        "name": "Bob Johnson",  
16        "score": 3,  
17        "ready": true  
18      }  
19    ],  
20    "settings": {  
21      "max_score": 11,  
22      "ball_speed": 300  
23    }  
24  }  
25 }
```

Usos de JSON

Más allá de APIs REST:

- Ficheros de configuración
- Almacenamiento de datos en disco
- Bases de datos NoSQL (MongoDB)
- Comunicación entre servicios
- WebSockets y tiempo real



Principios de Diseño REST

Todo es un Recurso

En REST, todo es un recurso identificable

- **Recurso:** Cualquier entidad o información con significado
- Cada recurso tiene una **URI única** que lo identifica
- Los recursos pueden ser individuales o colecciones

Ejemplos:

Recursos individuales:

```
/players/alice  
/matches/12345  
/leaderboard
```

Colecciones:

```
/players  
/matches  
/players/alice/scores
```



Principios de Diseño de URIs

Reglas de buenas prácticas:

Hacer:

- Usar sustantivos, no verbos
 - `/players/alice` ✓
- Usar plurales para colecciones
 - `/players` para lista
 - `/players/alice` para uno
- Crear jerarquías lógicas
 - `/players/alice/matches`

Evitar:

- Verbos en la URI
 - `/getPlayer?id=alice` ✗
- Nombres inconsistentes
 - Mezclar singular/plural
- URIs planas sin relación
- Mayúsculas y guiones bajos



Tip

Usar minúsculas y guiones: `/game-sessions/active`



Métodos HTTP

Los Cuatro Métodos Principales

Operaciones CRUD sobre recursos

Método	Operación	Seguro	Idempotente	Cacheable
GET	Obtener	✓	✓	✓
POST	Crear	✗	✗	✗
PUT	Actualizar	✗	✓	✗
DELETE	Eliminar	✗	✓	✗

Nota

Seguro: no modifica el servidor **Idempotente:** múltiples peticiones = mismo resultado

GET - Obtener Información

Características:

- Obtiene información sin modificar el servidor
- Seguro e idempotente
- Cacheable por navegadores y proxies

Ejemplos:

```
GET /players/alice HTTP/1.1  
→ Devuelve información del jugador alice
```

```
GET /matches HTTP/1.1  
→ Devuelve lista de todas las partidas
```

```
GET /leaderboard?limit=10 HTTP/1.1  
→ Devuelve top 10 del ranking
```

POST - Crear Recursos

Características:

- Crea nuevos recursos en el servidor
- El servidor decide el ID del recurso
- Devuelve el recurso creado en la respuesta
- No es seguro ni idempotente

Ejemplo:

```
POST /matches HTTP/1.1  
Content-Type: application/json
```

```
{  
  "type": "pong",  
  "mode": "ranked",  
  "player_id": "alice"  
}
```

```
→ 201 Created  
Location: /matches/67890
```



PUT - Actualizar Recursos

Características:

- Actualiza un recurso existente
- Envía el recurso completo (reemplazo)
- No es seguro pero sí idempotente
- Múltiples PUT producen el mismo resultado

Ejemplo:

```
PUT /players/alice HTTP/1.1  
Content-Type: application/json
```

```
{  
  "name": "Alice Smith",  
  "level": 16,  
  "score": 9000  
}
```

```
→ 200 OK
```



Nota



PATCH se usa para actualizaciones parciales (no cubierto aquí)

DELETE - Eliminar Recursos

Características:

- Elimina un recurso del servidor
- No es seguro pero sí idempotente
- Eliminar varias veces = mismo resultado (no existe)
- Responde con 204 (sin contenido) o 200

Ejemplo:

```
DELETE /matches/12345 HTTP/1.1
```

```
→ 204 No Content
```

Anatomía de Peticiones y Respuestas

Petición HTTP REST Completa

```
POST /matches HTTP/1.1
Host: api.game.com
Content-Type: application/json
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Length: 89
```

```
{
  "type": "pong",
  "mode": "ranked",
  "max_score": 11,
  "player_id": "alice"
}
```

Tres partes:

1. **Línea de petición:** Método, ruta y versión HTTP
2. **Headers:** Metadatos (tipo contenido, auth, etc.)
3. **Body:** Datos JSON enviados



Headers Comunes en Peticiones

Cabeceras importantes:

- `Content-Type: application/json`
 - Indica que enviamos JSON
- `Accept: application/json`
 - Indica que queremos recibir JSON
- `Authorization: Bearer <token>`
 - Token de autenticación
- `Content-Length: 89`
 - Tamaño del cuerpo en bytes



Respuesta HTTP REST Completa

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: http://api.game.com/matches/67890
Content-Length: 156
```

```
{
  "id": "67890",
  "type": "pong",
  "mode": "ranked",
  "max_score": 11,
  "status": "waiting",
  "created_at": "2025-01-15T14:30:00Z",
  "players": ["alice"]
}
```

Tres partes:

1. **Línea de estado:** Versión HTTP, código y mensaje
2. **Headers:** Metadatos de la respuesta
3. **Body:** Datos JSON del recurso



Headers Comunes en Respuestas

Cabeceras importantes:

- `Content-Type: application/json`
 - La respuesta es JSON
- `Location: http://api.game.com/matches/67890`
 - URI del recurso creado (en POST)
- `Content-Length: 156`
 - Tamaño de la respuesta en bytes



Códigos de Estado HTTP

Categorías de Códigos

Números de tres dígitos que comunican el resultado

Categoría	Significado	Uso
1xx	Informativas	Raramente usados
2xx	Éxito	Operación exitosa
3xx	Redirecciones	Acciones adicionales
4xx	Error del cliente	Error en la petición
5xx	Error del servidor	Fallo del servidor

1xx - Respuestas Informativas

Indican que la petición fue recibida y continúa

- Raramente usados en REST
- Más relevante:
 - **101 Switching Protocols**
 - Cambio de HTTP a WebSockets

Nota

Veremos el código 101 cuando estudiemos WebSockets



2xx - Respuestas Exitosas

La petición fue procesada correctamente

- **200 OK**
 - Éxito general en GET, PUT
 - Respuesta con contenido
- **201 Created**
 - Recurso creado tras POST
 - Debería incluir header `Location`
- **204 No Content**
 - Éxito sin contenido
 - Típico en DELETE



3xx - Redirecciones

El cliente debe tomar acciones adicionales

- **301 Moved Permanently**
 - Recurso movido permanentemente
 - Nueva ubicación en header `Location`
- **304 Not Modified**
 - Usado con caché
 - El recurso no ha cambiado desde última petición



Tip

Las redirecciones ayudan a mantener APIs retrocompatibles

4xx - Errores del Cliente

Error en la petición del cliente

- **400 Bad Request:** Petición mal formada
- **401 Unauthorized:** Autenticación requerida
- **403 Forbidden:** Sin permisos (aunque esté autenticado)
- **404 Not Found:** Recurso no existe
- **409 Conflict:** Conflicto con estado actual (ej: usuario ya existe)
- **422 Unprocessable Content:** Formato correcto pero errores de validación



5xx - Errores del Servidor

El servidor no pudo completar la petición

- **500 Internal Server Error**
 - Error genérico del servidor
 - Algo falló en el procesamiento
- **503 Service Unavailable**
 - Servidor temporalmente no disponible
 - Debe incluir `Retry-After`
- **504 Gateway Timeout**
 - Timeout esperando respuesta de otro servidor

Advertencia

Los errores 5xx indican problemas del servidor, no del cliente

Servidor con Node.js y Express

Express.js

Framework web minimalista para Node.js

Características:

- Sistema de enrutamiento robusto
- Soporte para middlewares
- Manejo de errores integrado
- Compatibilidad con todos los métodos HTTP
- Estándar de la industria

Instalación:

```
1 npm init -y  
2 npm install express@4
```



Configuración de Proyecto

Habilitar módulos ES6:

```
1 {  
2   "name": "mi-api-juego",  
3   "version": "1.0.0",  
4   "type": "module",  
5   "dependencies": {  
6     "express": "^4.18.0"  
7   }  
8 }
```

⚠ Importante

"type": "module" permite usar `import` en lugar de `require`

Estructura de Proyecto

```
mi-api-juego/  
├── node_modules/      # Dependencias (no subir a git)  
├── src/  
│   ├── controllers/  # Lógica de negocio  
│   ├── routes/       # Definición de rutas  
│   └── app.js         # Punto de entrada  
├── package.json  
└── package-lock.json
```



Servidor Básico con Express

```
1 import express from 'express';
2
3 const app = express();
4
5 // Middleware para parsear JSON
6 app.use(express.json());
7
8 // Iniciar servidor
9 app.listen(8080, () => {
10   console.log('Servidor ejecutándose en http://localhost:8080');
11 });
```

⚠ Importante

`express.json()` es fundamental para leer el body de las peticiones

Ejecutar:

```
1 node src/app.js
```



Servir Archivos Estáticos

Express puede servir HTML, CSS, JS del cliente

```
1 app.use(express.static('public'));
```

Estructura:

```
mi-api-juego/  
├── public/  
│   ├── index.html  
│   ├── script.js  
│   └── style.css  
└── src/  
    └── app.js
```

Acceso:

- <http://localhost:8080/index.html>
- <http://localhost:8080/script.js>



Ejemplo simple: Servidor POST

Servidor simple con endpoint POST:

```
1 import express from 'express';
2 const app = express();
3 app.use(express.json());
4
5 const messages = [];
6
7 // Endpoint POST para crear un mensaje
8 app.post('/messages', (req, res) => {
9   const { text, author } = req.body;
10  const newMessage = {
11    id: Date.now(), text, author,
12    timestamp: new Date().toISOString()
13  };
14  messages.push(newMessage);
15  res.status(201).json(newMessage);
16 });
17
18 app.get('/messages', (req, res) => {
19   res.json(messages);
20 });
21
22 app.listen(8080, () => {
23   console.log('Servidor en http://localhost:8080');
24 });
```



Invocar con fetch()

Cliente JavaScript usando POST:

```
1 async function createMessage() {  
2   const response = await fetch('http://localhost:8080/messages', {  
3     method: 'POST',  
4     headers: {  
5       'Content-Type': 'application/json'  
6     },  
7     body: JSON.stringify({  
8       text: 'Hola desde el cliente',  
9       author: 'Alice'  
10    })  
11  });  
12  
13  const newMessage = await response.json();  
14  console.log('Mensaje creado:', newMessage);  
15 }  
16  
17 createMessage();
```

Obtener los mensajes

```
1 const response = await fetch('http://localhost:8080/messages');  
2 const messages = await response.json();
```



Controladores

Funciones con lógica de negocio

- Procesan las peticiones HTTP
- Manipulan datos recibidos
- Devuelven respuestas apropiadas
- Separan lógica del enrutamiento

Patrón con closures para estado privado:

```
1  const createAnunciosController = () => {  
2    // Estado privado  
3    const anuncios = [];  
4    let nextId = 1;  
5  
6    const getAll = (req, res) => {  
7      res.json(anuncios);  
8    };  
9  
10   return { getAll };  
11 };  
12  
13 export default createAnunciosController;
```



Controlador Completo - Ejemplo

```
1 // src/controllers/anunciosController.js
2 const createAnunciosController = () => {
3   const anuncios = [];
4   let nextId = 1;
5
6   const getAll = (req, res) => {
7     res.json(anuncios);
8   };
9
10  const create = (req, res) => {
11    const { nombre, asunto, comentario } = req.body;
12
13    const nuevoAnuncio = {
14      id: nextId++,
15      nombre,
16      asunto,
17      comentario
18    };
19
20    anuncios.push(nuevoAnuncio);
21    res.status(201).json(nuevoAnuncio);
22  };
23
24  return { getAll, create };
25 };
```



Rutas

Conectan URLs con controladores

```
1 // src/routes/anunciosRoutes.js
2 import express from 'express';
3 import createAnunciosController from '../controllers/anunciosController.js';
4
5 const router = express.Router();
6 const controller = createAnunciosController();
7
8 router.get('/', controller.getAll);
9 router.post('/', controller.create);
10
11 export default router;
```

Nota

`router.get('/')` se convertirá en `GET /anuncios` con el prefijo

Integrar Rutas en la Aplicación

```
1 // src/app.js
2 import express from 'express';
3 import anunciosRoutes from './routes/anunciosRoutes.js';
4
5 const app = express();
6 app.use(express.json());
7
8 // Registrar rutas con prefijo
9 app.use('/anuncios', anunciosRoutes);
10
11 app.listen(8080);
```

Resultado:

- GET /anuncios → obtener todos
- POST /anuncios → crear nuevo



Operación GET - Recurso Específico

```
1 app.get('/anuncios/:id', (req, res) => {  
2   const { id } = req.params;  
3   const anuncio = anuncios.find(a => a.id === parseInt(id));  
4  
5   if (!anuncio) {  
6     return res.status(404).json({ error: 'No encontrado' });  
7   }  
8  
9   res.json(anuncio);  
10 });
```



Tip

`:id` es un parámetro dinámico disponible en `req.params.id`

Operación POST - Validación

```
1 app.post('/anuncios', (req, res) => {
2   const { nombre, asunto, comentario } = req.body;
3
4   // Validar datos
5   if (!nombre || !asunto) {
6     return res.status(400).json({
7       error: 'Nombre y asunto son requeridos'
8     });
9   }
10
11   const nuevoAnuncio = {
12     id: Date.now(),
13     nombre,
14     asunto,
15     comentario
16   };
17
18   anuncios.push(nuevoAnuncio);
19   res.status(201).json(nuevoAnuncio);
20 });
```

Operación PUT

```
1 app.put('/anuncios/:id', (req, res) => {
2   const { id } = req.params;
3   const { nombre, asunto, comentario } = req.body;
4
5   const anuncio = anuncios.find(a => a.id === parseInt(id));
6
7   if (!anuncio) {
8     return res.status(404).json({ error: 'No encontrado' });
9   }
10
11   // Actualizar propiedades
12   anuncio.nombre = nombre;
13   anuncio.asunto = asunto;
14   anuncio.comentario = comentario;
15
16   res.json(anuncio);
17 });
```

Operación DELETE

```
1 app.delete('/anuncios/:id', (req, res) => {  
2   const { id } = req.params;  
3   const index = anuncios.findIndex(a => a.id === parseInt(id));  
4  
5   if (index === -1) {  
6     return res.status(404).json({ error: 'No encontrado' });  
7   }  
8  
9   const anuncioEliminado = anuncios.splice(index, 1)[0];  
10  res.json(anuncioEliminado);  
11 });
```

Nota

También se puede devolver 204 (No Content) sin body

Middlewares

Funciones que procesan peticiones antes del controlador

Usos comunes:

- Logging de peticiones
- Autenticación y autorización
- Validación de datos
- Manejo de errores

Firma:

```
1 (req, res, next) => {  
2   // Procesar  
3   next(); // Pasar al siguiente middleware/ruta  
4 }
```

Middleware de Logging

```
1 app.use((req, res, next) => {  
2   console.log(`${req.method} ${req.path}`);  
3   next(); // Importante: pasar control  
4 });
```

Advertencia

Olvidar llamar a `next()` bloquea la petición

Middleware de Manejo de Errores

Firma especial con 4 parámetros

```
1 app.use((err, req, res, next) => {  
2   console.error(err.stack);  
3   res.status(500).json({  
4     error: 'Error interno del servidor'  
5   });  
6 });
```

⚠ Importante

Debe colocarse **después** de todas las rutas

Cliente JavaScript con fetch()

La API fetch()

API estándar moderna para peticiones HTTP

- Integrada en todos los navegadores modernos
- Utiliza Promises para operaciones asíncronicas
- Sintaxis limpia y potente

Dos formas de manejar respuestas:

Callbacks con .then()

```
1 fetch(url)
2   .then(response => response.json())
3   .then(data => console.log(data))
4   .catch(error => console.error(error));
```

async/await (recomendado)

```
1 async function getData() {
2   const response = await fetch(url);
3   const data = await response.json();
4   console.log(data);
5 }
```



async/await: Sintaxis Recomendada

Código más legible y natural

```
1  async function getPlayer() {  
2    try {  
3      const response = await fetch('https://api.game.com/players/alice');  
4  
5      if (!response.ok) {  
6        throw new Error('HTTP error ' + response.status);  
7      }  
8  
9      const data = await response.json();  
10     console.log(data);  
11  
12   } catch (error) {  
13     console.error('Error:', error);  
14   }  
15 }
```



Tip

Siempre usar try/catch para manejar errores

GET - Obtener Datos

Método por defecto de fetch()

```
1 // Obtener un jugador específico
2 async function getPlayer(id) {
3     const response = await fetch(`https://api.game.com/players/${id}`);
4     if (!response.ok) {
5         throw new Error('HTTP error ' + response.status);
6     }
7     return await response.json();
8 }
9
10 // Uso
11 const player = await getPlayer('alice');
12 console.log(player.name);
13 console.log(player.level);
```

Nota

Template literals (` `) permiten insertar variables en strings



POST - Crear Recurso

Especificar método, headers y body

```
1  async function createPlayer(data) {
2    const response = await fetch('https://api.game.com/players', {
3      method: 'POST',
4      headers: {
5        'Content-Type': 'application/json'
6      },
7      body: JSON.stringify(data)
8    });
9
10   if (!response.ok) {
11     throw new Error('HTTP error ' + response.status);
12   }
13   return await response.json();
14 }
15
16 // Uso
17 const newPlayer = await createPlayer({
18   username: 'charlie',
19   email: 'charlie@example.com'
20 });
21 console.log('Creado con ID:', newPlayer.id);
```



PUT - Actualizar Recurso

Reemplazar completamente el recurso

```
1  async function updatePlayer(id, player) {
2    const response = await fetch(`https://api.game.com/players/${id}`, {
3      method: 'PUT',
4      headers: {
5        'Content-Type': 'application/json'
6      },
7      body: JSON.stringify(player)
8    });
9
10   if (!response.ok) {
11     throw new Error('HTTP error ' + response.status);
12   }
13   return await response.json();
14 }
15
16 // Uso: se envía el recurso completo
17 await updatePlayer('charlie', {
18   username: 'charlie',
19   email: 'newemail@example.com',
20   level: 5
21 });
```

Nota

PUT reemplaza todo el recurso; PATCH actualiza solo campos específicos



DELETE - Eliminar Recurso

Método más simple

```
1 async function deletePlayer(id) {  
2     const response = await fetch(`https://api.game.com/players/${id}`, {  
3         method: 'DELETE'  
4     });  
5  
6     return response.ok;  
7 }  
8  
9 // Uso  
10 const eliminado = await deletePlayer('charlie');  
11 if (eliminado) {  
12     console.log('Jugador eliminado correctamente');  
13 }
```



Tip

`response.ok` es `true` para códigos 2xx

Manejo de Errores y Códigos de Estado

Manejo robusto con switch

```
1  async function fetchWithErrorHandling(url) {
2    try {
3      const response = await fetch(url);
4
5      if (response.ok) {
6        // 204 No Content no tiene body
7        return response.status === 204 ? null : await response.json();
8      }
9
10     switch (response.status) {
11       case 401:
12         throw new Error('No autenticado');
13       case 403:
14         throw new Error('Sin permisos');
15       case 404:
16         throw new Error('Recurso no encontrado');
17       case 500:
18         throw new Error('Error del servidor');
19       default:
20         throw new Error(`Error: ${response.status}`);
21     }
22   } catch (error) {
23     console.error('Error en petición:', error);
24     throw error;
25   }
```



Ventajas de la Arquitectura REST

Separación de Responsabilidades

Componentes independientes:

API REST:

- Operaciones de gestión
- Registro y autenticación
- Puntuaciones y estadísticas
- Perfiles y rankings

Sistema de Comandos:

- Lógica del juego
- Acciones de jugadores
- Estado del juego



Tip

Cada componente puede evolucionar independientemente

Escalabilidad

Escalar servicios según necesidades

- Operaciones de gestión (REST) tienen diferentes cargas
- Operaciones de tiempo real (WebSockets) requieren más recursos
- Podemos escalar cada servicio independientemente
- Optimización específica para cada tipo de comunicación



Testabilidad

Pruebas independientes:

- Probar API REST sin necesidad del juego completo
- Probar sistema de comandos sin conexión a red
- Herramientas específicas para cada tipo de prueba
- Tests automatizados más simples

Flexibilidad

La misma API sirve a múltiples clientes:

- Juego web (navegador)
- Aplicación móvil (iOS/Android)
- Herramientas de administración
- Servicios backend

Nota

Arquitectura preparada para futuras expansiones

Resumen

Conceptos Clave

API REST:

- Estilo arquitectónico basado en HTTP
- Operaciones CRUD sobre recursos
- URIs bien diseñadas (sustantivos, jerarquías)
- Métodos HTTP correctos (GET, POST, PUT, DELETE)
- Códigos de estado apropiados (2xx, 4xx, 5xx)

JSON:

- Formato estándar de intercambio
- Ligero y legible
- Objetos y arrays anidados



Implementación

Cliente JavaScript:

- API `fetch()` con `async/await`
- Manejo de errores robusto
- Validación de códigos de estado

Servidor Node.js:

- Express.js como framework
- Controladores con lógica de negocio
- Rutas para organizar endpoints
- Middlewares para funcionalidad transversal

